Ingénierie des données pour le machine learning

Principes opérationnels

Université de Montpellier – LIRMM

HAI817I

Pascal Poncelet
Pascal.Poncelet@lirmm.fr
http://www.lirmm.fr/~poncelet

Plan

1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Plan

1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Ingénierie des données : objectif du cours

- Objectif : passer de données brutes à un tableau prêt pour l'entraînement.
- On vise des règles simples et réutilisables : mêmes étapes sur données d'entraînement et sur nouvelles données.
- Le pré-traitement doit s'appliquer automatiquement : pas de "correction à la main" en production.

Sorties attendues :

- un tableau final prêt pour entraîner un modèle,
- un pipeline reproductible qui applique les mêmes transformations à de nouvelles données.

Rappel (cours précédent) :

- ordre de grandeur : souvent 80% du travail est sur les données.

De brut à prêt pour l'entraînement

Données brutes :

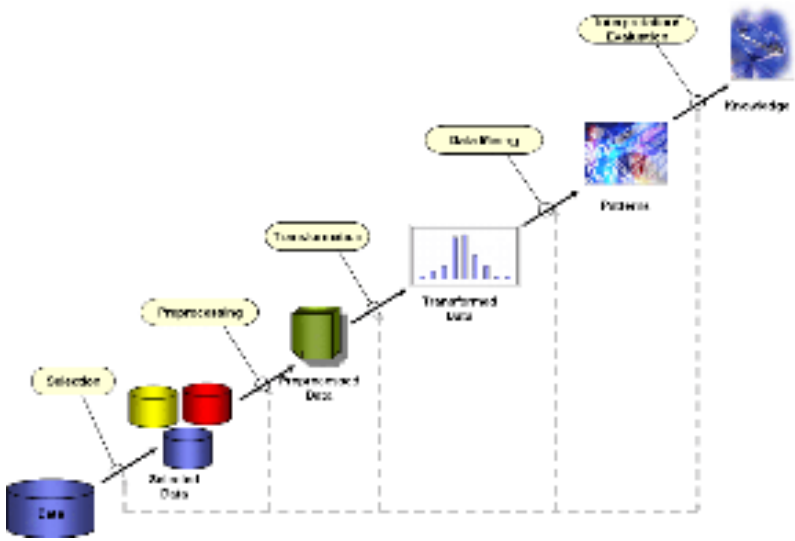
Species	Island	Sex	Culmen Length (mm)	Body Mass (g)
Adelie	Biscoe	MALE	39.1	3750
Adelie	Dream		40.3	3800
Gentoo	Biscoe	FEMALE	50.0	5000
Chinstrap	Dream	MALE		3700

Table finale :

Species	Culmen Len_z	Sex MALE	Sex FEMALE	Sex Unknown	Island Biscoe	Island Dream	BodyMass Cat
Adelie	-0.62	1	0	0	1	0	léger
Adelie	-0.48	0	0	1	0	1	léger
Gentoo	1.10	0	1	0	1	0	lourd
Chinstrap	0.00	1	0	0	0	1	léger

Processus : on itère et chaque choix a des conséquences

- Un projet démarre par une question et une sortie attendue.
- On revient souvent sur les étapes d'ingénierie : on corrige, on améliore, on recommence.
- Les choix d'ingénierie des données conditionnent la suite :
  - types et unités,
  - manquants,
  - encodage,
  - mise à l'échelle,
  - agrégations temporelles.
- Attention : rendre ces choix explicites et reproductibles.



Source : Wikimedia Commons

Plan

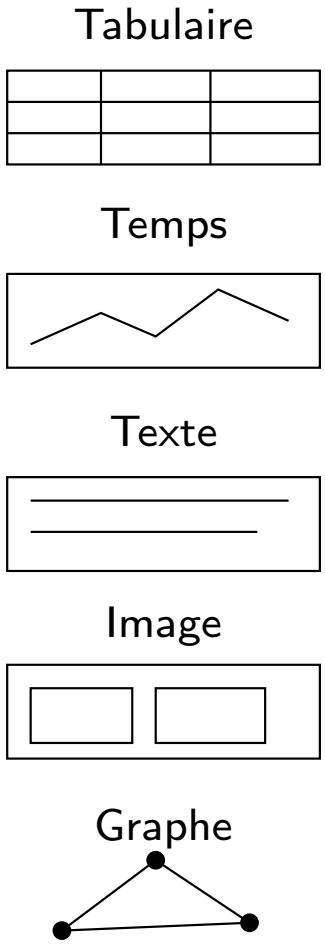
1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Bien comprendre ses données

- Avant de nettoyer et transformer, on doit comprendre ce qu'on a.
- Objectif : repérer rapidement ce qui va poser problème.
- Outils : types, statistiques descriptives, visualisations simples.

Types de jeux de données utilisés en machine learning

- Tabulaire : lignes / colonnes (ex. clients)
- Série temporelle : mesures dans le temps (ex. capteurs)
- Texte : documents, messages (ex. support)
- Image : classification, détection (ex. défauts)
- Graphe : relations (ex. liens entre comptes)



Objets, attributs, types : le vocabulaire minimal

- Un jeu de données décrit des objets (une entité).
  - ventes : un client, une transaction
  - médical : un patient, un séjour
  - université : un étudiant, un cours
- En tabulaire : une ligne = un objet, une colonne = un attribut (ou feature ou dimension).
- Un attribut a un type et ce type impose des traitements possibles.

Exemples de types :

- numérique, catégoriel, date/temps, texte

Types de données : décisions différentes

- Numérique : âge, montant, durée
- Catégoriel : ville, source, produit
- Date/temps : date d'achat, timestamp
- Texte : commentaire, description

- Exemple : `code_postal` ressemble à un nombre, mais se traite comme une catégorie.

colonne	type	remarque
age	numérique	quantité
code_postal	catégoriel	identifiant
date_achat	date/temps	ordre
source	catégoriel	modalités
commentaire	texte	libre

Jeu de données fil conducteur : Palmer Penguins

- `Palmer Penguins`.
- 344 manchots, 3 espèces (Adelie, Chinstrap, Gentoo).
- Variables : mesures (bec, nageoire, masse) + contexte (île, sexe, année).
- Objectif : comprendre les données avant de choisir un modèle.

Source : Allison Horst

Pandas : manipuler des données tabulaires en Python

- Pandas est une bibliothèque Python pour manipuler des tables de données.
- Objet central : le `DataFrame` (lignes + colonnes).
- Entrées courantes : CSV, Excel, JSON, fichiers texte, SQL.
- Très utilisé car il permet de :
  - explorer vite (tri, filtres, statistiques),
  - nettoyer et transformer (types, manquants, catégories),
  - préparer une table prête pour le machine learning.

Source : Allison Horst

DataFrame : un aperçu rapide

- Commande : `df.head()`

...

	studyName	Sample Number	Species	Region	Island	Stage	Individual ID	Clutch Completion	Date Egg	Culmen Length (mm)	Culmen Depth (mm)	Flipper Length (mm)	Body Mass (g)	Sex	Delta 15 N (o/oo)	Delta 13 C (o/oo)
0	PAL0708	1	Adelie Penguin (Pygoscelis adeliae)	Anvers	Torgersen	Adult, 1 Egg Stage	N1A1	Yes	2007-11-11	39.1	18.7	181.0	3750.0	MALE	NaN	NaN
1	PAL0708	2	Adelie Penguin (Pygoscelis adeliae)	Anvers	Torgersen	Adult, 1 Egg Stage	N1A2	Yes	2007-11-11	39.5	17.4	186.0	3800.0	FEMALE	8.94956	-24.69454
2	PAL0708	3	Adelie Penguin (Pygoscelis adeliae)	Anvers	Torgersen	Adult, 1 Egg Stage	N2A1	Yes	2007-11-16	40.3	18.0	195.0	3250.0	FEMALE	8.36821	-25.33302
3	PAL0708	4	Adelie Penguin (Pygoscelis adeliae)	Anvers	Torgersen	Adult, 1 Egg Stage	N2A2	Yes	2007-11-16	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	PAL0708	5	Adelie Penguin (Pygoscelis adeliae)	Anvers	Torgersen	Adult, 1 Egg Stage	N3A1	Yes	2007-11-16	36.7	19.3	193.0	3450.0	FEMALE	8.76651	-25.32426

Source : exécution locale (notebook)

Voir les types et repérer les manquants

- Commande : `df.info()`

```
...  <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 344 entries, 0 to 343
     Data columns (total 17 columns):
      #   Column               Non-Null Count  Dtype
     ---  ------               --------------  -----
      0   studyName            344 non-null    object
      1   Sample Number        344 non-null    int64
      2   Species              344 non-null    object
      3   Region               344 non-null    object
      4   Island               344 non-null    object
      5   Stage                344 non-null    object
      6   Individual ID        344 non-null    object
      7   Clutch Completion    344 non-null    object
      8   Date Egg             344 non-null    object
      9   Culmen Length (mm)   342 non-null    float64
      10  Culmen Depth (mm)    342 non-null    float64
      11  Flipper Length (mm)  342 non-null    float64
      12  Body Mass (g)        342 non-null    float64
      13  Sex                  333 non-null    object
      14  Delta 15 N (o/oo)    330 non-null    float64
      15  Delta 13 C (o/oo)    331 non-null    float64
      16  Comments             54 non-null     object
     dtypes: float64(6), int64(1), object(10)
     memory usage: 45.8+ KB
```

Source : exécution locale (notebook)

- Types fréquents : `int64` (entiers), `float64` (réels), `object` (texte ou catégories).
- Manquants : `non-null` indique combien de valeurs sont présentes par colonne.

Catégories : compter les modalités

- Commande : `df["Species"].value_counts()`

...

Species	count
Adelie Penguin (Pygoscelis adeliae)	152
Gentoo penguin (Pygoscelis papua)	124
Chinstrap penguin (Pygoscelis antarctica)	68

dtype: int64

Source : exécution locale (notebook)

- Utile pour repérer : catégories rares, catégories dominantes, fautes de saisie (variantes).

Manquants : mesurer rapidement

- Commande : `df.isna().mean().sort_values(ascending=False)`

...

	0
Comments	0.843023
Delta 15 N (o/oo)	0.040698
Delta 13 C (o/oo)	0.037791
Sex	0.031977
Culmen Length (mm)	0.005814
Flipper Length (mm)	0.005814
Culmen Depth (mm)	0.005814
Body Mass (g)	0.005814
studyName	0.000000
Date Egg	0.000000
Clutch Completion	0.000000
Individual ID	0.000000
Stage	0.000000
Island	0.000000
Region	0.000000
Species	0.000000
Sample Number	0.000000

dtype: float64

- 0.05 = 5% de valeurs manquantes dans la colonne.

Numériques : résumer les ordres de grandeur

- Commande : `df.describe()`

...

	Sample Number	Culmen Length (mm)	Culmen Depth (mm)	Flipper Length (mm)	Body Mass (g)	Delta 15 N (o/oo)
count	344.000000	342.000000	342.000000	342.000000	342.000000	330.000000
mean	63.151163	43.921930	17.151170	200.915205	4201.754386	8.733382
std	40.430199	5.459584	1.974793	14.061714	801.954536	0.551770
min	1.000000	32.100000	13.100000	172.000000	2700.000000	7.632200
25%	29.000000	39.225000	15.600000	190.000000	3550.000000	8.299890
50%	58.000000	44.450000	17.300000	197.000000	4050.000000	8.652405
75%	95.250000	48.500000	18.700000	213.000000	4750.000000	9.172123
max	152.000000	59.600000	21.500000	231.000000	6300.000000	10.025440

Source : exécution locale (notebook)

- À repérer : min/max, médiane (50%), écart-type, valeurs suspectes.

Statistiques : attention aux interprétations

- Une moyenne peut masquer la réalité des individus.
  - « The average human has one breast and one testicle. »
- Attribution : Des MacHale.

Source : Wikimedia Commons (Joe Wos)

Anscombe : mêmes statistiques, formes différentes

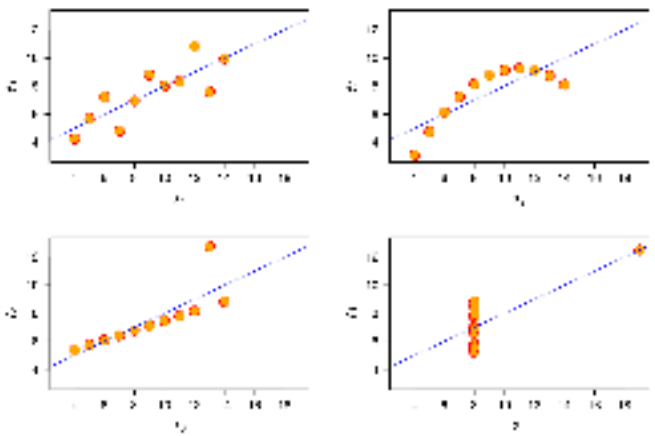
Quartet d'Anscombe

I		II		III		IV	
x	*y*	*x*	*y*	*x*	*y*	*x*	*y*
10,0	8,04	10,0	9,14	10,0	7,46	8,0	6,58
8,0	6,95	8,0	8,14	8,0	6,77	8,0	5,76
13,0	7,58	13,0	8,74	13,0	12,74	8,0	7,71
9,0	8,81	9,0	8,77	9,0	7,11	8,0	8,84
11,0	8,33	11,0	9,26	11,0	7,81	8,0	8,47
14,0	9,96	14,0	8,10	14,0	8,84	8,0	7,04
6,0	7,24	6,0	6,13	6,0	6,08	8,0	5,25
4,0	4,26	4,0	3,10	4,0	5,39	19,0	12,50
12,0	10,84	12,0	9,13	12,0	8,15	8,0	5,56
7,0	4,82	7,0	7,26	7,0	6,42	8,0	7,91
5,0	5,68	5,0	4,74	5,0	5,73	8,0	6,89

Source : Wikipedia

Propriété	Valeur
Moyenne des *x*	9,0
Variance des *x*	10,0
Moyenne des *y*	7,5
Variance des *y*	3,75
Coefficient de corrélation entre les *x* et les *y*	0,816
Équation de la droite de régression linéaire	$y=3+0,5\,x$
Somme des carrés des erreurs relativement à la moyenne	110,0

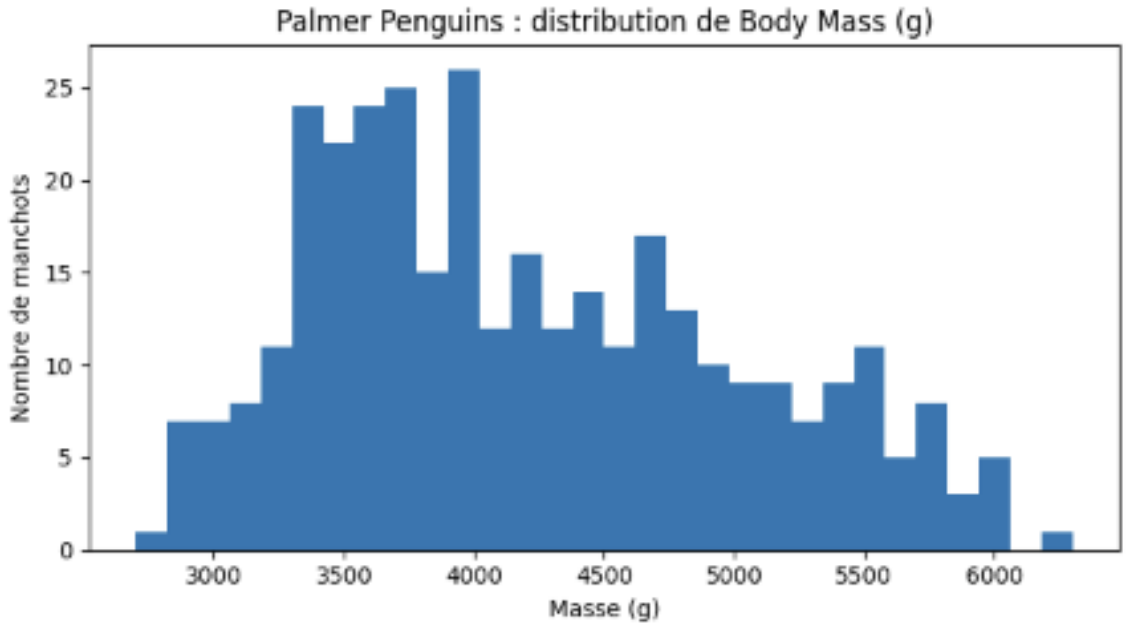
Source : Wikipedia



Source : Wikimedia Commons

Distribution : histogramme

- Commande (Pandas) : `df["Body Mass (g)"].hist(bins=30)`

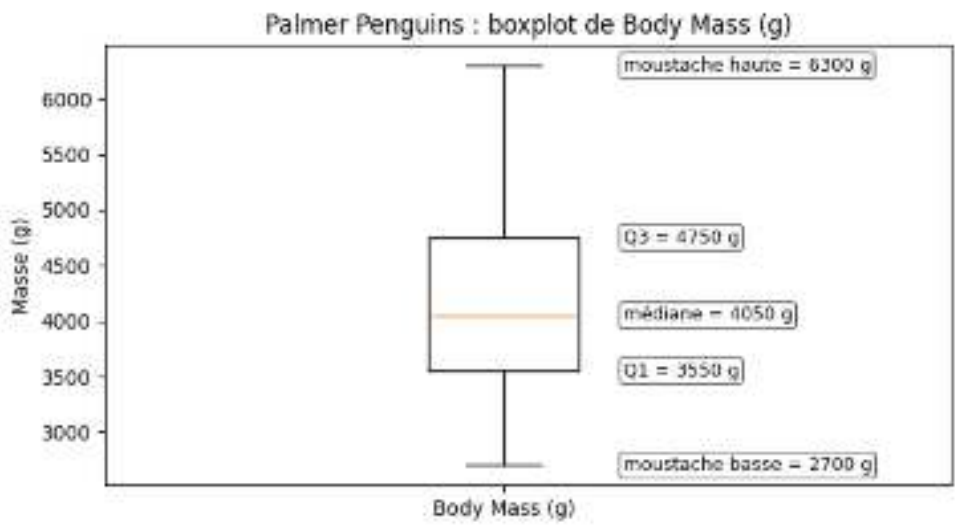


Source : exécution locale (notebook) via Matplotlib

- Intérêt : voir la forme de la distribution (symétrie, étalement, valeurs rares, possibles outliers).
- `bins=30` : on découpe l'intervalle des valeurs en 30 classes ; plus de classes = plus fin (mais plus bruité), moins de classes = plus lissé (mais moins détaillé).

Boxplot : repérer dispersion et valeurs atypiques

- Commande : `df[["Body Mass (g)"]].boxplot()`



Source : exécution locale (notebook) via Matplotlib

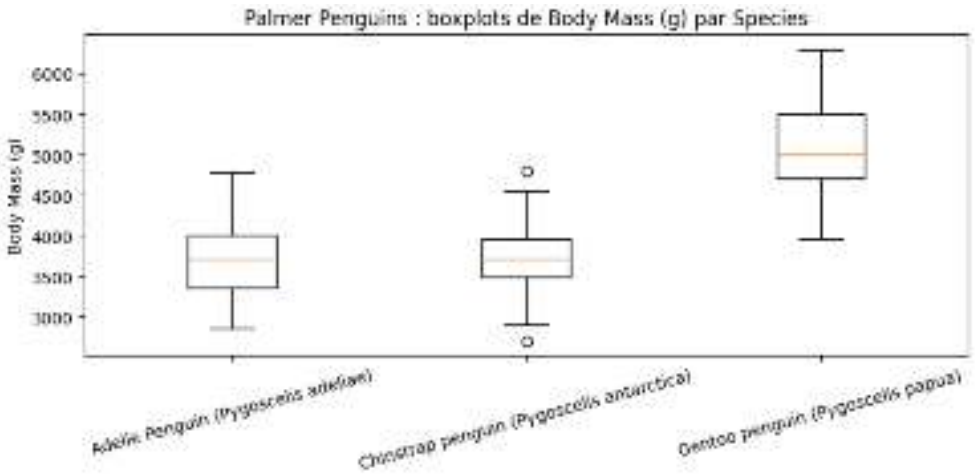
- Lecture (ici `Body Mass (g)`, n=342) :

Q1,Q3 = [3550, 4750] g (IQR = 1200 g), médiane = 4050 g

  - moustaches : 2700 g et 6300 g
  - règle 1.5×IQR : bornes [1750, 6550] g, ici 0 valeur atypique

Boxplot : comparer par espèce

- Commande : `df.boxplot(column="Body Mass (g)", by="Species")`



Source : exécution locale (notebook) via Matplotlib

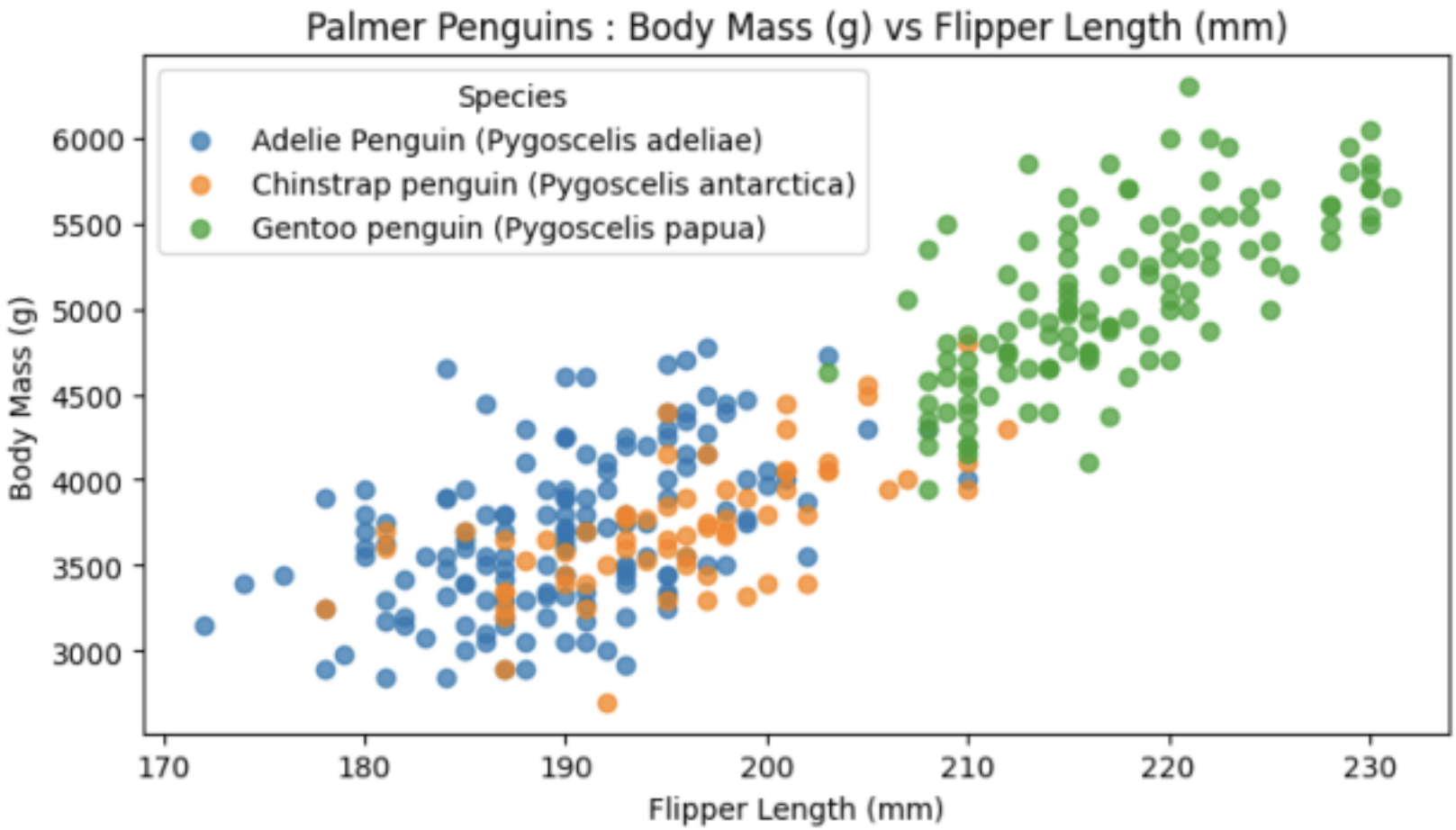
- Remarque (ex. `Chinstrap`, n=68) :

Q1,Q3 = [3487.5, 3950.0] g (IQR = 462.5 g), médiane = 3700.0 g

  - moustaches = 2900.0 g et 4550.0 g
  - valeurs atypiques (1.5×IQR) : bornes [2793.75, 4643.75] g, ici 2 points (2700.0 g et 4800.0 g)

Relation entre deux variables : nuage de points

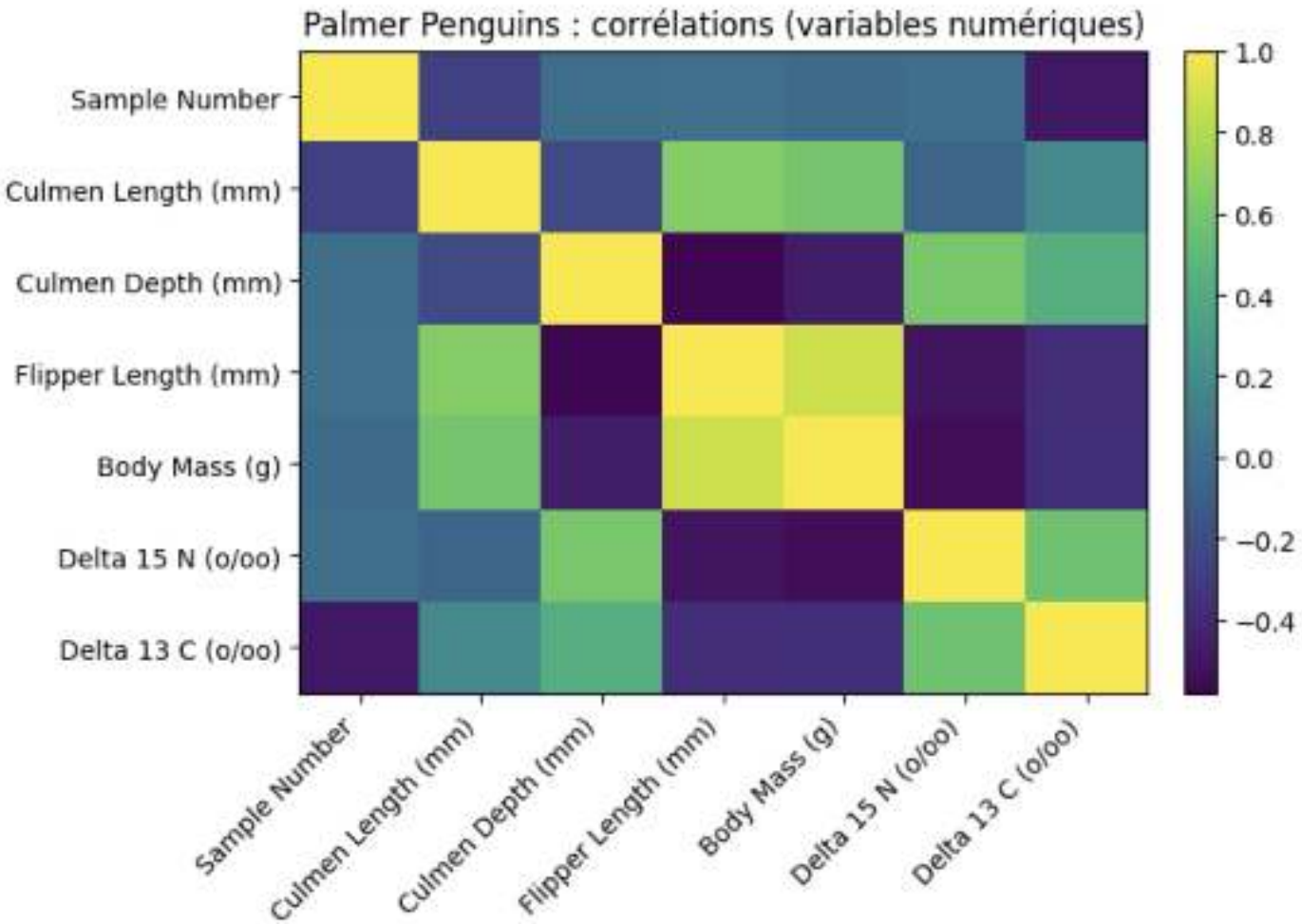
- Commande : `scatter (Body Mass (g) vs Flipper Length (mm)) par 'Species'`



Source : exécution locale (notebook) via Matplotlib

Vue d'ensemble : corrélations entre variables numériques

- Commande : `corr() + heatmap (variables numériques)`



Source : exécution locale (notebook) via Matplotlib

Plan

1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Qualité des données : contrôles rapides

- Avant de transformer, on vérifie que les données sont cohérentes.
- Objectif : repérer tôt les problèmes qui polluent l'entraînement.

- Contrôles typiques : types, bornes, manquants, doublons, catégories.
- Sortie attendue : quelques règles explicites (quoi corriger, quoi exclure).

Schéma minimal : type + rôle (quoi en faire)

- Idée : écrire une fois ce que chaque colonne représente et si elle sert au modèle.

colonne	type	rôle	remarque
Species	catégorie	cible / classe	3 espèces
Island	catégorie	entrée	modalités stables
Sex	catégorie	entrée	manquants possibles
Body Mass (g)	numérique	entrée	valeurs plausibles
Culmen Length (mm)	numérique	entrée	valeurs positives
Culmen Depth (mm)	numérique	entrée	valeurs positives
Flipper Length (mm)	numérique	entrée	valeurs positives
Date Egg	date	contexte	format uniforme
Year	entier	contexte	attention si redondant
Individual ID	identifiant	à exclure	ne pas apprendre un ID
Comments	texte libre	à exclure	non structuré (ici)

- Réflexe : identifiants, texte libre, colonnes constantes = souvent hors-modèle (au moins au début).

Catégories : repérer les variantes et incohérences

- Commande : `df["Species"].value_counts(dropna=False)`

...

Species	count
Adelie Penguin (Pygoscelis adeliae)	152
Gentoo penguin (Pygoscelis papua)	124
Chinstrap penguin (Pygoscelis antarctica)	68

dtype: int64

Source : exécution locale (notebook)

- À repérer : variantes (`"adelie"`, `"Adelie "`, etc.), catégories dominantes, catégories rares, éventuels `NaN`.

Manquants : comparer par groupe

- Commande :
  `df.groupby("Species")[cols_focus].apply(lambda g: g.isna().mean())`

Species	Comments	Delta 15 N (o/oo)	Delta 13 C (o/oo)	Sex	Culmen Length (mm)	Flipper Length (mm)	Culmen Depth (mm)	Body Mass (g)
Adelie Penguin (Pygoscelis adeliae)	0.83	0.07	0.07	0.04	0.01	0.01	0.01	0.01
Chinstrap penguin (Pygoscelis antarctica)	0.78	0.01	0.00	0.00	0.00	0.00	0.00	0.00
Gentoo penguin (Pygoscelis papua)	0.90	0.02	0.02	0.04	0.01	0.01	0.01	0.01

Source : exécution locale (notebook)

- Lecture rapide :
  - `Comments` est quasi vide dans toutes les espèces : colonne peu exploitable telle quelle.
  - Certains manquants sont spécifiques à un groupe (ex. isotopes surtout chez *Adelie*).
- Intérêt : un problème peut être concentré sur un sous-groupe.

Doublons : repérer des lignes identiques

- Commande : `df.duplicated().sum()`

```
...  Nombre de doublons (lignes identiques) : 0
```

Source : exécution locale (notebook)

Valeurs impossibles : bornes simples

- Idée : une règle métier simple vaut mieux qu'un modèle qui "s'adapte".
- Exemples de contrôles :
  - `Body Mass (g)` : strictement positif
  - `Culmen Length (mm)` : strictement positif
  - `Date Egg` : format de date uniforme
- Commande : `df[df["Body Mass (g)"] <= 0]`

Plan

1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Valeurs manquantes : ce que ça peut vouloir dire

- Un manquant n'a pas toujours la même cause.
  - inconnu : l'info existe mais n'a pas été renseignée (ex. `Sex`)
  - non applicable : l'info n'a pas de sens pour cet objet
  - bug / collecte : capteur, formulaire, import, saisie
- Réflexe : avant de "remplir", se demander pourquoi c'est manquant.
- Décision attendue (par colonne) : supprimer / imputer / garder manquant + indicateur.

Manquants : voir vite les colonnes concernées

- Commande :
  `df[cols].isna().sum().sort_values(ascending=False)`

...

	0
Sex	11
Culmen Length (mm)	2
Flipper Length (mm)	2
Culmen Depth (mm)	2
Body Mass (g)	2
Species	0

dtype: int64

Source : exécution locale (notebook)

- On traite d'abord les colonnes "utiles au modèle" pas les colonnes qu'on exclut.

Valeurs manquantes : 4 stratégies (à choisir selon le contexte)

- 1) Supprimer (lignes / colonnes) : simple, mais peut jeter beaucoup d'information.
- 2) Imputer (moyenne, médiane, mode) : rapide, souvent une première baseline.
- 3) Catégorie dédiée : `"Unknown"` pour une variable catégorielle.
- 4) Prédire les manquants : on estime la valeur à partir des autres colonnes (ex. K-NN).

- Idée : choisir la stratégie selon la colonne et l'objectif.

Stratégie 1 : supprimer (simple, mais parfois coûteux)

- Option A — supprimer des lignes (quand elles sont inexploitables) :
  - ex. : lignes où toutes les *mesures* sont manquantes (`Culmen...`, `Flipper...`, `Body Mass (g)`).
  - commande : `df = df[ df[core].isna().all(axis=1)]`

```
... Nb lignes où toutes les mesures sont manquantes : 2
```

	Species	Island	Sex	Date Egg	Culmen Length (mm)	Culmen Depth (mm)	Flipper Length (mm)	Body Mass (g)
3	Adelie Penguin (Pygoscelis adeliae)	Torgersen	NaN	2007-11-16	NaN	NaN	NaN	NaN
271	Gentoo penguin (Pygoscelis papua)	Biscoe	NaN	2009-12-01	NaN	NaN	NaN	NaN

Source : exécution locale (notebook)

- Option B — supprimer une colonne (trop vide / peu utile au début) :
  - ex. : `Comments` (texte libre, très manquant, non structuré ici)
  - commande : `df = df.drop(columns=["Comments"])`
- Risque : perdre beaucoup d'exemples ou introduire un biais si les manquants ne sont pas aléatoires.

Stratégie 2 : imputer (rapide, bonne baseline)

- Numérique : moyenne ou médiane (souvent médiane si outliers).
- Catégoriel : mode (valeur la plus fréquente).
- Commandes :

```
m = df["Body Mass (g)"].median()
df["Body Mass (g)"] = df["Body Mass (g)"].fillna(m)
```

- Limite : on “aplatit” la variabilité (et on sous-estime l’incertitude).

Stratégie 3 : catégorie dédiée (simple, souvent efficace)

- Idée : pour une variable catégorielle, remplacer `NaN` par une modalité explicite.
- Exemple : `Sex` : `"Unknown"` (au lieu de supprimer des lignes).
- Commandes :

```
df["Sex"] = df["Sex"].fillna("Unknown")
```

- Intérêt : on garde les lignes et on rend le manquant visible au modèle.
- Limite : `"Unknown"` peut regrouper des causes différentes (non mesuré, non renseigné, etc.).

Stratégie 4 : prédire les valeurs manquantes (plus coûteux)

- Idée : traiter la colonne avec des `NaN` comme une cible, et utiliser les autres colonnes comme variables explicatives.
- Exemple : prédire `Sex` manquant à partir des mesures et du contexte.
- Principe :
  - ici la cible y n'est plus `Species` : c'est la colonne à compléter (ex. `Sex`),
  - on entraîne sur les lignes où `Sex` est connue (lignes complètes pour y),
  - on prédit `Sex` uniquement là où elle manque,
  - on réinjecte les prédictions dans la colonne `Sex`.
- Il faut bien évaluer l'erreur sinon on ajoute du bruit "propre".

Plan

1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Pré-traitements : ordre recommandé

- 1) Catégories : transformer en numérique (sans inventer de faux ordre).
- 2) Variables continues : discrétiser si on veut des classes ou des règles simples.
- 3) Échelle : rendre les mesures comparables.

- Exemple : `Island`, `Sex` (catégories), `Body Mass (g)` (continue).

Catégories : le bon réflexe

- Problème : un modèle attend des nombres, pas des chaînes de caractères.
- Objectif : transformer une catégorie en numérique sans ajouter de sens artificiel.

- Dans Penguins, `Island` et `Sex` n'ont pas d'ordre naturel.

Catégories ordonnées : encoder avec un ordre (map ou LabelEncoder)

- Objectif : remplacer des catégories par des entiers en respectant un ordre connu (ex. tailles : S < M < L < XL).
- Option A (recommandée) : mapping explicite

```
replace_map = {'XL': 4, 'L': 3, 'M': 2, 'S': 1}
df[col] = df[col].map(replace_map)
```

- Option B : `LabelEncoder` (possible, mais il faut contrôler la correspondance)

```
le = LabelEncoder()
df[col] = le.fit_transform(df[col])
```

  - vérifier `le.classes_` pour savoir quel entier correspond à quelle catégorie
- Ne pas appliquer à `Island` ou `Sex` (pas d'ordre).

Catégories sans ordre : OneHotEncoder

- Problème : `LabelEncoder` donne des entiers, donc une comparaison numérique possible (ex. $2 > 1$) qui n'a aucun sens si la catégorie n'est pas ordonnée.
- Solution : One Hot Encoding (une colonne par modalité, valeurs 0/1).
- Commandes :

```
ohe = OneHotEncoder(sparse_output=False,
handle_unknown="ignore")
X_ohe = ohe.fit_transform(df[["Island","Sex"]])
```

	Island_Biscoe	Island_Dream	Island_Torgersen	Sex_FEMALE	Sex_MALE	Sex_Unknown
0	0.0	0.0	1.0	0.0	1.0	0.0
1	0.0	0.0	1.0	1.0	0.0	0.0
2	0.0	0.0	1.0	1.0	0.0	0.0
3	0.0	0.0	1.0	0.0	0.0	1.0
4	0.0	0.0	1.0	1.0	0.0	0.0

Source : exécution locale (notebook)

- Beaucoup de modalités implique beaucoup de colonnes.

Discrétiser une variable continue : pourquoi faire ?

- Objectif : transformer une mesure continue en classes (intervalle de valeurs).
- Utile pour :
  - produire des règles simples (faible / moyen / élevé),
  - visualiser ou résumer rapidement une distribution,
  - créer une variable catégorielle exploitable par certains modèles ou analyses.
- Exemple : discrétiser `Body Mass (g)` ou `Flipper Length (mm)`.

Discrétiser `Body Mass (g)` : combien de classes ?

- Objectif : remplacer une valeur continue (2700–6300 g) par quelques catégories.
- Observation (par espèce, médianes) :
  - *Adelie* : 3700 g *Chinstrap* : 3700 g *Gentoo* : 5000 g

...

Species	count	mean	median	std	min	max
Adelie Penguin (Pygoscelis adeliae)	151	3701	3700	459	2850	4775
Gentoo penguin (Pygoscelis papua)	123	5076	5000	504	3950	6300
Chinstrap penguin (Pygoscelis antarctica)	68	3733	3700	384	2700	4800

Source : exécution locale (notebook)

- Choix simple ici : 2 classes (`"léger"` vs `"lourd"`) car *Gentoo* est nettement au-dessus.
- Alternative : 3 classes (`"faible"`, `"standard"`, `"élevée"`) si l'on veut des règles plus fines.
- Attention avec deux classes : recouvrement important entre *Adelie* et *Chinstrap*.

Discrétiser Body Mass (g) : avec pd.cut

- Commande : `pd.cut` pour créer des classes à partir d'intervalles.
- Exemple (2 classes) :
  - bornes : `[2700, 4300, 6300]` (seuil vers 4300 g)
  - étiquettes : `["léger","lourd"]`
- Code :

```
bins = [2700, 4300, 6300]
labels = ["léger","lourd"]
df["BodyMass_Cat"] = pd.cut(df["Body Mass (g)"],
bins=bins, labels=labels, include_lowest=True)
```

- Penser à vérifier le résultat :

```
df["BodyMass_Cat"].value_counts(dropna=False)
```

Attention aux distances !

- Beaucoup de méthodes comparent des individus via une distance.
- Exemples : k-plus proches voisins, k-means, SVM (RBF), LOF, etc.
- Problème : si une variable a une grande échelle, elle peut dominer la distance.

Exemple : une variable peut dominer la distance

- Deux manières de comparer deux manchots :
  - `Body Mass (g)` (milliers)
  - `Culmen Length (mm)` (dizaines)
- Exemple :
  - Pingouin A : 4000 g et 45 mm
  - Pingouin B : 4300 g et 48 mm
  - Différences : 300 g et 3 mm
- Distance euclidienne :
  - $\sqrt{300^2 + 3^2} \approx 300.0$
  - La contribution de 3 mm devient négligeable face à 300 g.

Distances pour attributs numériques

- Euclidienne (L2) : distance "géométrique" classique.

$$d_2(x, y) = \sqrt{\sum_{f=1}^{p} (x_f - y_f)^2}$$

- Manhattan (L1) : somme des écarts absolus.

$$d_1(x, y) = \sum_{f=1}^{p} |x_f - y_f|$$

- Chebyshev (L∞) : plus grand écart sur une dimension.

$$d_\infty(x, y) = \max_f |x_f - y_f|$$

Minkowski : une famille qui généralise

- Minkowski (L_h) :

$$d_h(x, y) = \left(\sum_{f=1}^{p} |x_f - y_f|^h \right)^{1/h}$$

- Cas particuliers :
  - $h = 1$: Manhattan
  - $h = 2$: Euclidienne
  - h très grand : proche de Chebyshev
- Plus h est grand, plus on pénalise fortement les gros écarts.

Distances pour attributs catégoriels

- Pour une variable catégorielle, l'idée la plus simple est :

$$d(x, y) = \begin{cases} 0 & \text{si } x = y \\ 1 & \text{si } x \neq y \end{cases}$$

- Exemple :
  - `Island` : même île $\Rightarrow$ distance 0, îles différentes $\Rightarrow$ distance 1
  - `Sex` : même valeur $\Rightarrow$ 0, différente $\Rightarrow$ 1

Données binaires : Hamming et Jaccard

- Hamming (vecteurs binaires) : nombre de positions différentes

$$d_H(x, y) = \sum_{f=1}^{p} 1[x_f \neq y_f]$$

- Jaccard (présence/absence) : compare surtout les 1

$$J(x, y) = \frac{|x \cap y|}{|x \cup y|}, \qquad d_J(x, y) = 1 - J(x, y)$$

- Utilité : après OneHotEncoding, on peut comparer `Island` et `Sex` proprement.

Distance cosinus : utile en haute dimension (texte, embeddings)

- Idée : comparer l'orientation de deux vecteurs plutôt que leur norme.

$$\cos(\theta) = \frac{x \cdot y}{\|x\| \, \|y\|}, \qquad d_{\cos}(x, y) = 1 - \cos(\theta)$$

- Adapté aux représentations type TF-IDF ou embeddings (normes peu informatives).

Données mixtes : une distance cohérente (Gower)

- Problème : on veut comparer deux individus avec des variables de natures différentes (mesures en mm/g, catégories comme `Island` et `Sex`).
- Idée : calculer une petite distance par colonne, toutes entre 0 et 1, puis faire une moyenne :
  - numérique : distance normalisée

$$d_f(x, y) = \frac{|x_f - y_f|}{\max(x_f) - \min(x_f)} \in [0, 1]$$

  - catégorielle : distance binaire

$$d_f(x, y) = \begin{cases} 0 & \text{si même catégorie} \\ 1 & \text{sinon} \end{cases}$$

- Distance finale : moyenne des d_f sur les variables comparables (et non manquantes).

Normalisation : rendre les mesures comparables

- Pourquoi : les distances (L1/L2, k-NN, k-means, SVM RBF, LOF, etc.) sont dominées par les variables à grande échelle.
- Objectif : mettre les variables numériques sur une échelle comparable.
- Deux choix classiques :
  - Min-max : valeurs dans $[0, 1]$

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

  - Standardisation : moyenne 0, écart-type 1

$$x' = \frac{x - \mu}{\sigma}$$

Normalisation : code minimal

- Standardiser (moyenne 0, écart-type 1) :

```
from sklearn.preprocessing import StandardScaler
num_cols = ["Culmen Length (mm)","Culmen Depth (mm)",
                 "Flipper Length (mm)","Body Mass (g)"]
scaler = StandardScaler()
X_num = scaler.fit_transform(df[num_cols])
```

- Alternative : Min-max (dans [0,1]) avec `MinMaxScaler`.

Normalisation : la faire maintenant ?

- Réponse : pas sur tout le jeu de données si l'objectif est de faire une classification ensuite !
- Pourquoi :
  - la normalisation calcule des statistiques (moyenne, écart-type, min/max),
  - si ces statistiques utilisent aussi le futur jeu de test, on introduit une fuite de données,
  - conséquence : l'évaluation devient trop optimiste (biais).

Plan

1. Introduction
2. Bien comprendre ses données
3. Qualité des données
4. Valeurs manquantes
5. Pré-traitements (catégories, discrétisation, échelle)
6. Bonnes pratiques

Bonne pratique : regrouper les pré-traitements

- Pendant l'exploration, on teste plusieurs règles (exclure, compléter, encoder, discrétiser).
- Risque : des règles dispersées dans le notebook, difficiles à relire et à réutiliser.
- Bonne pratique : regrouper ces règles dans un préprocesseur unique (une classe), avec une interface claire `fit/transform`.
- Intérêt : on applique exactement les mêmes décisions à de nouvelles données, sans réécrire du code.

Bonne pratique : décisions explicites par colonne

colonne	décision	exemple
Sex	catégorie dédiée si manquant	NaN transformé en "Unknown"
Island	One Hot Encoding	1 colonne par île
Body Mass (g)	(option) discrétisation	"léger"/"lourd"
Culmen Length (mm)	numérique	garder tel quel(ou standardiser plus tard)
Comments	exclure	texte libre quasi vide

- On sait exactement quoi faire et on peut modifier une règle sans casser le reste.

Scikit-learn : un préprocesseur réutilisable (TransformerMixin)

- Idée : écrire un objet compatible scikit-learn (hérite de `BaseEstimator` et `TransformerMixin`).
- `fit(X)` : mémorise ce qui doit être appris (ex. modalités pour OneHotEncoder).
- `transform(X)` : applique les règles (ex. `Unknown`, discrétisation, OHE, suppression).
- Résultat : un composant réutilisable, qui s'insère naturellement dans les outils scikit-learn.

Exemple : classe de pré-traitement (constructeur)

```python
class PenguinsPreprocessor(BaseEstimator, TransformerMixin):
    """
    Pré-traitements :
    - Exclure Comments (si présent)
    - Sex : NaN -> "Unknown"
    - Discrétiser Body Mass (g) en "léger"/"lourd"
    - OneHotEncoder sur Island et Sex (car non ordonnés)
    - Conserver les numériques tels quels (standardisation plus tard)
    """
    def __init__(
        self,
        drop_comments: bool = True,
        add_bodymass_cat: bool = True,
        bodymass_bins=(2700, 4300, 6300),
        bodymass_labels=("léger", "lourd"),
        ohe_cols=("Island", "Sex"),
        numeric_cols=("Culmen Length (mm)",
                      "Culmen Depth (mm)",
                      "Flipper Length (mm)",
                      "Body Mass (g)"),
    ):
        self.drop_comments = drop_comments
        self.add_bodymass_cat = add_bodymass_cat
        self.bodymass_bins = list(bodymass_bins)
        self.bodymass_labels = list(bodymass_labels)
        self.ohe_cols = list(ohe_cols)
        self.numeric_cols = list(numeric_cols)
        self._ohe = None
```

Exemple : apprendre ce qui doit l'être (fit)

```python
def fit(self, X: pd.DataFrame, y=None):
    X = X.copy()
    # 1) Exclure Comments
    if self.drop_comments and "Comments" in X.columns:
        X = X.drop(columns=["Comments"])
    # 2) Sex : NaN -> "Unknown"
    if "Sex" in X.columns:
        X["Sex"] = X["Sex"].fillna("Unknown")
    # 3) Colonnes catégorielles réellement disponibles pour OHE
    self._ohe_cols_in_ = [c for c in self.ohe_cols if c in X.columns]
    # 4) OneHotEncoder appris sur train (fit)
    if len(self._ohe_cols_in_) > 0:
        self._ohe = OneHotEncoder(
            sparse_output=False,
            handle_unknown="ignore")
        self._ohe.fit(X[self._ohe_cols_in_])
        self._ohe_feature_names_ = list(
            self._ohe.get_feature_names_out(self._ohe_cols_in_)
    else:
        self._ohe = None
        self._ohe_feature_names_ = []
    # 5) Numériques à conserver (standardisation plus tard)
    self._numeric_cols_in_ = [c for c in self.numeric_cols if c in X.columns]
    return self
```

Exemple : appliquer les règles (transform)

```python
def transform(self, X: pd.DataFrame):
    X = X.copy()
    # 1) Exclure Comments
    if self.drop_comments and "Comments" in X.columns:
        X = X.drop(columns=["Comments"])
    # 2) Sex : NaN -> "Unknown"
    if "Sex" in X.columns:
        X["Sex"] = X["Sex"].fillna("Unknown")
    # 3) (Option) Discrétisation : BodyMass_Cat
    if self.add_bodymass_cat and "Body Mass (g)" in X.columns:
        X["BodyMass_Cat"] = pd.cut(
            X["Body Mass (g)"],
            bins=self.bodymass_bins,
            labels=self.bodymass_labels,
            include_lowest=True)
    # 4) One Hot Encoding sur Island/Sex (transform seulement)
    if self._ohe is not None and len(self._ohe_cols_in_) > 0:
        X_ohe = self._ohe.transform(X[self._ohe_cols_in_])
        df_ohe = pd.DataFrame(X_ohe, columns=self._ohe_feature_names_,
                              index=X.index)

        # on retire les colonnes d'origine encodées puis on concatène
        X = pd.concat([X.drop(columns=self._ohe_cols_in_), df_ohe], axis=1)

    # 5) Conserver les numériques tels quels
    return X
```

Exemple d'utilisation : fit_transform

```
# -------------------------
# Exemple d'utilisation
# -------------------------
prep = PenguinsPreprocessor()
df_ready = prep.fit_transform(df)
```

Des questions ?